# Partie 2: communiquer à partir de données

**Lino Galiana**
Version du 20 octobre 2026

Les *data scientists* doivent être en mesure de synthétiser l'information présente dans un jeu de données par le biais de la représentation graphique, car le cerveau humain comprend mieux les informations à travers des figures que des tableaux. La visualisation de données est importante à la fois dans une démarche exploratoire pour comprendre la structure des phénomènes étudiés mais aussi dans une phase de communication de résultats à des publics n'ayant pas forcément accès à la donnée brute et devant se contenter de synthèses. Cette partie du cours est une introduction à ce vaste sujet par le biais de la pratique à travers la construction de graphiques descriptifs et de cartes.

---

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

## Table des matières

**1 Introduction** ..... **2**
1.1 La visualisation des données, une part essentielle du travail de communication ..... 2
1.2 La place de la visualisation dans le processus de valorisation de la donnée ..... 2
**2 Communiquer, une ouverture au *data storytelling*** ..... **3**
**3 Communiquer, une ouverture à la mise à disposition d'applications** ..... **5**
**4 L'écosystème `Python`** ..... **5**
4.1 Les *packages* de visualisations de données ..... 5
4.2 Les applications de visualisation ..... 6
4.3 Résumé de cette partie ..... 7
4.4 Références utiles ..... 8
Informations additionnelles ..... 8
**Bibliographie** ..... **10**

Compétences à l'issue de ce chapitre

- Reconnaître l'importance de la visualisation des données à la fois pour l'exploration et la communication des résultats ;
- Comprendre le rôle clé de la datavisualisation dans le processus de valorisation de la donnée, du signal dans un jeu de données au message synthétisé ;
- Savoir identifier les enjeux cognitifs liés à la représentation graphique pour favoriser la compréhension — rendre le complexe lisible et engageant ;
- Connaître les grandes familles de bibliothèques Python dédiées à la visualisation :
  - bibliothèques pour visualisation statique (Matplotlib, Seaborn, Plotnine…) ;
  - bibliothèques pour visualisation interactive (Plotly, Folium…) ;
- Être capable de reproduire des graphiques à partir d'exemples réels (notamment issus des données open-data de la ville de Paris) pour appréhender la pratique et ses choix ;
- Appréhender les principes de base de la communication visuelle : sélectionner le type de graphique adapté (distribution, corrélation, série temporelle, carte…), penser au message principal à transmettre.

# 1 Introduction

Une partie essentielle du travail du *data scientist* consiste à synthétiser l'information que contient ses jeux de données afin de distinguer ce qui relève du signal, sur lequel il pourra se concentrer, et ce qui relève du bruit inhérent à tout jeu de données. Dans le travail du *data scientist*, lors d'une phase exploratoire, il y a donc un aller-retour constant entre information synthétique et jeu de données désagrégé. Il est ainsi essentiel de savoir synthétiser l'information dans un jeu de données avant d'en saisir la structure, cette dernière pouvant ensuite guider les exploitations ultérieures, pour une phase de modélisation ou de correction de données (détection d'anomalies ou de mauvaises remontées de données).

Nous avons déjà exploré une partie essentielle de ce travail, à savoir la construction de statistiques descriptives pertinentes et fiables. Néanmoins, si on se contentait de présenter l'information en utilisant des sorties brutes issues du combo `groupby` et `agg` sur un *DataFrame* `Pandas`, notre connaissance des données serait assez limitée. La mise en oeuvre de tableaux stylisés à partir de `Great Tables` constituait déjà un progrès dans cette démarche mais, en vérité, notre cerveau se représente l'information de manière beaucoup plus intuitive par le biais de visualisations graphiques simples que dans un tableau.

## 1.1 La visualisation des données, une part essentielle du travail de communication

En tant qu'humains, nos capacités cognitives étant limitées, nous ne pouvons appréhender qu'une information limitée là où l'ordinateur est capable de traiter de grands volumes d'information. En tant que *data scientist*, cela signifie qu'utiliser nos compétences informatiques et statistiques pour obtenir des représentations synthétiques de nos nombreux jeux de données est essentiel pour être en mesure de répondre à nos besoins opérationnels ou scientifiques. L'ensemble des méthodes et des outils qui constituent la boîte à outil des *data scientists* vise à simplifier l'appréhension puis l'exploitation de jeux de données dont le volume dépasse nos capacités cognitives.

Ceci nous entraîne vers la question de la visualisation des données, un ensemble d'outils et de principes pour représenter de manière synthétique des faits stylisés ou contextualiser une donnée individuelle. La visualisation de données est l'art et la science de **représenter visuellement des informations complexes et abstraites à l'aide d'éléments visuels**. Son objectif principal est de synthétiser l'information présente dans un ensemble de données afin de faciliter la compréhension des enjeux de celle-ci pour une analyse ultérieure. La visualisation de données permet, entre autres, de mettre en évidence des tendances, des corrélations ou des anomalies qui pourraient être difficiles voire impossibles à saisir simplement en examinant des données brutes, ces dernières nécessitant une certaine mise en contexte pour porter du sens.

La visualisation de données joue un rôle crucial dans le processus d'analyse de données en fournissant des moyens visuels pour explorer, interpréter et communiquer des informations. Elle facilite la communication entre experts de la données, décideurs et grand public, en permettant aux derniers de bénéficier du travail rigoureux des premiers pour donner sens aux données sans la connaissance des subtilités conceptuelles qui ont permis de synthétiser l'information contenue dans celle-ci.

## 1.2 La place de la visualisation dans le processus de valorisation de la donnée

La visualisation des données n'est pas restreinte à la phase finale d'un projet, c'est à dire à la phase de communication de résultats à une audience qui n'a pas accès à la donnée ou n'a pas les moyens de la valoriser. La visualisation intervient à toutes les étapes du processus de valorisation de la donnée. Il s'agit d'ailleurs d'un travail essentiel pour trouver comment basculer de l'enregistrement,

un instantané d'un phénomène, à une donnée, un enregistrement qui a une valeur parce qu'il porte une information en tant que telle ou lorsqu'il est combiné avec d'autres enregistrements.

Le travail quotidien du *data scientist* consiste à regarder un jeu de données sous toutes ses coutures pour identifier les axes prioritaires d'extraction de valeur. Savoir rapidement quelles statistiques représenter, et comment, est essentiel pour gagner du temps sur cette partie exploratoire. Il s'agit principalement d'un travail de communication envers soi-même qui peut se permettre d'être brouillon car il s'agit de dégrossir le travail avant de polir certains angles. L'enjeu à ce niveau du processus est de ne pas manquer une dimension qui pourrait être porteuse de valeur.

Le travail de communication réellement chronophage intervient plutôt lorsqu'on communique à une audience ayant un accès limité à des données, ne connaissant pas bien les sources, ayant un temps d'attention limité ou n'ayant pas des compétences quantitatives. Ces publics ne peuvent se satisfaire d'une sortie brute comme un *DataFrame* dans un *notebook* ou un graphique produit en quelques secondes avec la méthode `plot` de `Pandas`. Il convient de s'adapter à leurs attentes, qui évoluent, et aux outils qu'ils connaissent, d'où la place de plus en plus importante prise par les sites web de *data visualisations*.

# 2 Communiquer, une ouverture au *data storytelling*

La visualisation de données a ainsi une place à part dans l'ensemble des techniques de la *data science*. Elle intervient à tous les stades du processus de production de la donnée, de l'amont (analyse exploratoire) à l'aval (restitution à des publics multiples) et peut, si elle est bien construite, permettre de saisir de manière intuitive la structure des données ou les enjeux de son analyse.

Art de la synthèse, la visualisation de données est également l'art de raconter une histoire et peut même, lorsqu'elle est bien construite, prétendre au rang de production artistique. La *dataviz* est un métier en soi dont on trouve de plus en plus de praticiens dans les titres de presse ou dans des entreprises spécialisées (`Datawrapper` par exemple).

Sans prétendre construire des visualisations aussi riches que celles des spécialistes, tout *data scientist* se doit d'être en mesure de pouvoir produire rapidement quelques visualisations permettant de synthétiser l'information présente dans les jeux de données à sa disposition. Une visualisation claire et lisible tout en restant simple peut être meilleure qu'un discours pour faire passer un message.

De même qu'un discours, une visualisation est une communication pour laquelle un locuteur - la personne construisant la visualisation - cherche à transmettre une information à un récepteur - éventuellement la même personne que le locuteur puisqu'une visualisation peut être construite pour soi-même dans une analyse exploratoire. Il n'est donc pas surprenant qu'à l'époque où la sémiologie occupait une part importante dans les débats intellectuels, notamment autour de la figure de Roland Barthes, le concept de sémiologie graphique ait émergé autour de la personne de Jacques Bertin [1], [2]. Cette approche permet de réfléchir sur la pertinence des techniques mises en oeuvre pour transmettre un message graphique et de nombreuses visualisations, si elles suivaient quelques-unes de ces règles, pourraient être améliorées à peu de frais.

Eric Mauvière, statisticien français héritier de l'école de la sémiologie graphique de Bertin, propose d'excellents contenus sur le sujet. Certaines des présentations qu'il a pu faire, notamment celle pour le `SSPHub` présentées dans la Note 1 devraient être visionnées dans toutes les formations de *data science* tant elles évoquent les nombreux écueils rencontrés par les *data scientists*.

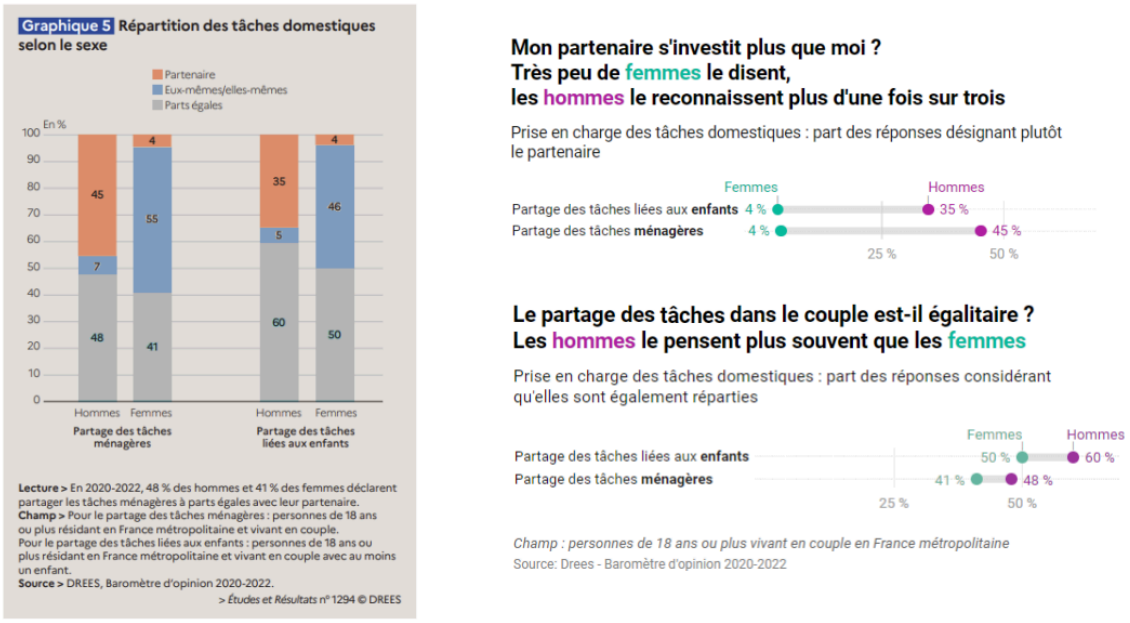



Figure 1. – Un exemple de deux visualisations faites sur le même jeu de données par Eric Mauvière, voir Note 1

**i** Note 1: Une conférence d'Eric Mauvière sur le sujet

```
//| echo: false
html`${slides_button}`
```

https://minio.lab.sspcloud.fr/lgaliana/ssphub/replay/20240229-dataviz-mauviere/video1991622347.mp4

```
//| echo: false
slides = "https://minio.lab.sspcloud.fr/lgaliana/ssphub/replay/20240229-dataviz-mauviere/conf_ssphub_icem7.pdf"
```

```
//| echo: false
slides_button = html`<p class="text-center">
  <a class="btn btn-primary btn-lg cv-download" href="${slides}" target="_blank">
    <i class="fa-solid fa-file-arrow-down"></i> Télécharger les slides
  </a>
</p>`
```

Comme le rappellent les travaux de recherche en sciences cognitives, la perception des graphiques est une opération cognitive complexe qui consiste à appréhender des relations quantitatives (hiérarchie, intensité…) à partir de signaux visuels. Des facteurs innés - par exemple une meilleure capacité à transposer des positions que des couleurs en information quantitative - et des facteurs culturels - la familiarité à différentes formes de dataviz - vont avoir un effet sur la compréhension d'une figure. Le cours au collège de France de Stanislas Dehaene sur la perception des graphiques évoque ces enjeux et revient sur l'histoire de la visualisation de données. A ce sujet, la bibliothèque de l'Insee a organisé une exposition en 2025 avec quelques visualisations qui valent le détour et apparaissent d'une étonnante modernité.

# 3 Communiquer, une ouverture à la mise à disposition d'applications

L'objectif de ce cours est d'introduire aux principaux outils et à la démarche que doivent adopter les *data scientists* face à divers jeux de données. Il devient néanmoins de plus en plus commun pour les *data scientists* de développer et mettre à disposition des applications interactives proposant un certain nombre d'explorations et de visualisations automatisées de données. Il s'agit d'enjeux plus avancés que ce cours mais qui constituent souvent un point d'entrée vers la *data science* pour des publics proches des *data scientists*, notamment les *data engineers*, les *data analysts* ou les statisticiens.

Nous évoquerons certains des outils privilégiés pour faire cela, notamment les écosystèmes liés aux applications *web* et aux outils `Javascript`. Ce besoin, devenu assez standard pour les *data scientists*, fait la passerelle avec la mise en production, l'enjeu principal d'un cours de 3e année de l'ENSAE construit par Romain Avouac et moi-même (site web du cours ensae-reproductibilite.github.io/). Le présent site web, par exemple, est construit selon ce principe grâce à des outils permettant d'exécuter de manière reproductible du `Python` sur des serveurs standardisés et ensuite mettre à disposition ce code par le biais d'un site web.

# 4 L'écosystème `Python`

Pour revenir à notre cours, nous présenterons dans cette partie quelques librairies et visualisations basiques en `Python` permettant de partir sur de bonnes bases. Les ressources pour approfondir et progresser dans l'art de la visualisation ne manquent pas, comme cet ouvrage [3].

## 4.1 Les *packages* de visualisations de données

L'écosystème `Python` pour la visualisation de données est très riche et très éclaté. Il est possible de consacrer des livres entiers à celui-ci [4]. `Python` propose de nombreuses librairies pour produire de manière rapide et relativement simple des visualisations de données[1].

Les librairies graphiques se distinguent principalement en deux familles:

- Les librairies de **représentations figées**. Celles-ci ont plutôt vocation à être intégrées dans des publications figées type PDF ou documents texte. Nous présenterons principalement `Matplotlib` et `Seaborn` mais il en existe d'autres, en pleine émergence, comme `Plotnine`, l'adaptation de `ggplot2` à l'écosystème `Python`.
- Les librairies de **représentations réactives**. Celles-ci sont adaptées à des représentations *web* et offrent la possibilité aux lecteurs d'agir sur la représentation graphique affichée. Les librairies qui proposent ces fonctionnalités reposent généralement sur `JavaScript`, l'écosystème du développement *web*, pour lequel elles offrent un point d'entrée via `Python`. Nous évoquerons principalement `Plotly` et `Folium` dans cette famille mais il existe de nombreux autres *frameworks* dans ce domaine[2].

[1] Pour être honnête, pendant longtemps `Python` a été sur ce point un peu moins agréable que `R` qui bénéficie de l'incontournable librairie `ggplot2`.

N'étant pas construite sur la grammaire des graphiques, la principe librairie de graphiques en `Python` qu'est `Matplotlib` est plus fastidieuse à utiliser que `ggplot2`.

`seaborn`, que nous présenterons, facilite un peu le travail de représentation graphique mais, là encore, il est difficile de faire plus malléable et universel que `ggplot2`.

La librairie `plotnine` vise à proposer une implémentation similaire à `ggplot` pour les utilisateurs de `Python`. Son développement est à suivre.

Il est tout à fait possible de faire des visualisations sophistiquées avec une chaine de bout en bout `Python` puisqu'il s'agit d'un langage couteau-suisse dont l'écosystème est très riche. Néanmoins, `Python` n'est pas la panacée et il peut parfois être utile, pour obtenir un produit fini parfaitement poli, de finaliser le travail avec d'autres langages, comme `Javascript` pour les visualisations réactives ou `QGIS` pour le travail cartographique. Ce cours donnera les outils minimums pour faire un travail rapide et plaisant mais le diable étant dans les détails, il ne faut pas s'arcbouter à vouloir utiliser `Python` pour tout et n'importe quoi.

Dans le domaine de la visualisation, ce cours adopte le parti pris d'explorer quelques librairies centrales à partir d'un nombre restreint d'exemples en répliquant des graphiques qu'on peut trouver sur le site d'*open data* de la mairie de Paris. La meilleure école pour la visualisation restant la pratique sur des jeux de données, il est recommandé d'explorer la richesse de l'écosystème de l'*open data* pour expérimenter des visualisations.

## 4.2 Les applications de visualisation

Cette partie du cours se focalise sur des représentations synthétiques simples. Elle n'évoque pas (*encore ?*) la construction d'applications de visualisation de données où un ensemble de graphiques se mettent à jour de manière synchrone en fonction d'actions d'utilisateurs.

Ceci dépasse en effet le cadre d'un cours d'introduction car construire ces applications impliquent de maîtriser des concepts plus complexes comme l'interaction entre une page *web* et un serveur, d'avoir des rudiments de connaissance en `Linux`, etc. Les concepts nécessaires à la compréhension de ces outils sont au coeur du cours de 3e année « Mise en production de projets de *data science* » que Romain Avouac donnons en 3e année d'ENSAE.

Néanmoins, comme la valorisation de données sous une forme applicative est très commune, il il est utile *a minima* d'évoquer la dualité entre sites statiques et applications dynamiques afin de donner les bons gestes et pointer vers les outils adéquat. Dans le monde de l'applicatif, il est important de distinguer le *front* (la page visible par les utilisateurs de l'application) du *back office* (le moteur qui effectue des actions en fonction des paramètres choisis par l'utilisateur de la page).

Il existe principalement deux paradigmes pour faire interagir ces deux éléments. La distinction principale entre ces deux approches est qu'elles s'appuient sur des serveurs différents. Un site statique repose sur un serveur web là où `Streamlit` s'appuie sur serveur classique en *backend*. La différence principale entre ces deux types de serveurs réside principalement dans leur fonction et leur utilisation:

- Un serveur *web* est spécifiquement conçu pour stocker, traiter et livrer des pages web (le *front*) aux clients. Cela inclut des fichiers HTML, CSS, JavaScript, images, etc. Les serveurs web écoutent les requêtes HTTP/HTTPS provenant des navigateurs des utilisateurs et y répondent en envoyant les données demandées. Cela n'empêche pas d'avoir des étapes complexes de valorisation de données, ni de la réactivité en embarquant du `Javascript` dans l'application mais les étapes de traitement en `Python` sont faites en amont de la mise à disposition de l'application. Pour les utilisateurs de `Python`, il existe plusieurs constructeurs de sites statiques avant une mise à disposition par le biais d'un hébergement sur `Github Pages`. Les deux écosystèmes les plus communs sont `Quarto Markdown` et `Django`, le premier étant plus simple d'usage et de maintenance que le second. Ce site, par exemple, est construit grâce à `Quarto` ce qui assure la reproductibilité des exemples présentés et une mise en forme ergonomique et paramétrable des résultats.
- Un serveur *backend* classique est conçu pour effectuer des opérations en réponse à un *front*, en l'occurrence une page *web*. Dans le contexte d'une application construite avec `Python`, il s'agit d'un serveur avec l'environnement `Python` *ad hoc* pour exécuter le code nécessaire à répondre

[2] A cet égard, je recommande vivement de suivre l'actualité de la *dataviz* sur la plateforme `Observable` qui tend à rapprocher les communautés des spécialistes de la *dataviz* et des analystes de données. La librairie `Plot` pourrait devenir un nouveau standard dans les prochaines années, sorte d'intermédiaire entre `ggplot` et `d3`.

à toute action d'un utilisateur de l'application. Le code est exécuté à la volée et non une fois pour toute comme dans l'approche précédente. Il s'agit donc d'un paradigme pouvant permettre plus de complexité applicative mais représentant un défi supplémentaire lors de la phase de mise en production. Dans l'écosystème `Python`, les deux principaux outils permettant de construire de telles applications sont `Streamlit` et `Dash`, le premier étant plus rapide à mettre en oeuvre que le second. Plus récemment, l'écosystème équivalent dominant en `R`, `Shiny` a été adapté en `Python` par `Posit`.

> **i** Fait-on toujours du tkinter ?
>
> Les écosystèmes présentés ci-dessus pour les applications réactives sont des *frameworks web*. Ils se distinguent des clients lourds comme `tkinter`, l'outil historique pour faire des interfaces graphiques. Outre l'aspect plus rudimentaire des interfaces `tkinter` par rapport à celles de `Streamlit`, `Dash` ou `Shiny`, il existe des raisons fortes pour privilégier ces derniers à `tkinter`.
>
> Ce dernier est un client lourd. Autrement dit, il est adhérent à un système d'exploitation et à des installations de *packages* en amont du fonctionnement de l'interface. Il est bien sûr possible de rendre portable celle-ci mais, comme cela est développé dans le cours de mise en production, il y a de nombreuses raisons pour lesquelles cette approche peut provoquer des erreurs ou des *bugs* inattendus. Les *frameworks web* présentent l'intérêt de simplifier cette mise à disposition en dissociant le *front* (des pages HTML et du CSS) du *back* (du code `Python`). Ils se sont donc imposés naturellement même si on retrouve encore beaucoup de ressources en ligne datées sur le développement d'applications avec `tkinter`.

En ce qui concerne la construction d'applications, le premier réflexe à avoir est: *« ai-je besoin de faire une application réactive ou un site statique ne suffit-il pas ? »*. Ce dernier étant beaucoup plus facile à mettre en oeuvre et ayant une charge de maintenance minimale, c'est souvent un choix rationnel. S'il devient complexe de faire un site statique, par exemple parce qu'ils impliquent des calculs sophistiqués qu'il serait complexe de mettre en oeuvre sans compétences `JavaScript`, on peut alors se poser la question de la séparation entre *front* et *back* en reportant les calculs vers une API, construite par exemple par le biais de `FastAPI`. Il s'agit, par exemple, d'une méthode pratique pour mettre à disposition un modèle de *machine learning* comme le dernier chapitre de la partie modélisation l'évoquera. Si la mise en oeuvre d'une API est compliquée ou bien est un bazooka pour tuer une mouche, alors on pourra aller vers une application réactive du type de `Streamlit`.

Encore une fois, la construction d'une application fait appel à des concepts qui dépassent un niveau introductif en `Python`. Avoir conscience des bons réflexes peut néanmoins faire économiser un temps non négligeable en évitant de patauger dans la semoule à cause d'un mauvais choix initial.

## 4.3 Résumé de cette partie

Pour en revenir au contenu de cette partie après cet *aparté*, celle-ci est divisée en deux et chaque chapitre est lui-même dual, selon qu'on s'intéresse aux représentations figées ou dynamiques :

- Dans un premier temps, nous évoquerons des représentations graphiques standards (histogrammes, diagrammes en barre...) pour synthétiser certaines informations quantitatives ;
  - Les représentations fixes reposeront sur `Pandas`, `Matplotlib` et `Seaborn`
  - Les graphiques réactifs s'appuieront sur `Plotly`
- Dans un deuxième temps, nous présenterons les représentations cartographiques:
  - Les cartes figées à partir de `Geopandas` ou de `plotnine`
  - Les cartes réactives avec `Folium` (adaptation `Python` de la librairie `Leaflet.js`)

## 4.4 Références utiles

La visualisation de données est un art qui s'apprend, au début, principalement par la pratique. Néanmoins, il n'est pas évident de produire des visualisations lisibles et ergonomiques et il est utile de s'inspirer d'exemples de spécialistes (les grands titres de presse disposent d'excellentes visualisations).

Voici quelques ressources utiles sur ces sujets :

- `Datawrapper` propose un excellent blog sur les bonnes pratiques de visualisation, notamment avec les articles de Lisa Charlotte Muth. Je recommande notamment cet article sur les couleurs ou celui-ci sur les textes ;
- Le blog d'Eric Mauvière ;
- *« La Sémiologie graphique de Jacques Bertin a cinquante ans »* ;
- Les visualisations *trending* sur `Observable` ;
- Le *New York Times* (les rois de la *dataviz*) revient tous les ans sur les meilleures visualisations de l'année dans la veine du *data scrollytelling*. Voir par exemple la rétrospective de l'année 2022.

> Quelques ressources sur Streamlit ou Dash
>
> Outre le cours de 3e année de l'ENSAE, le lab de *data science* de l'Insee a construit de nombreux tutoriels pour s'approprier les écosystèmes d'applications réactives en `Python` qui sont l'un des produits les plus attractifs de l'écosystème `Python`.
>
> Voici par exemple un tutoriel 101 très détaillé sur `Streamlit` permettant de créer une application type `Yuka` sur les données de l'`openfoodfacts`. Un autre tutoriel pas à pas construit par l'Insee est consacré à `streamlit` et vise à proposer la construction d'un tableau de bord du trafic aérien.

Et quelques références supplémentaires, citées dans cette introduction :

# Informations additionnelles

> Environnement Python
>
> Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).
>
> L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
```

```
    "graphviz==0.20.3",
    "great-tables>=0.12.0",
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```



Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

# Bibliographie


[1] J. Bertin, *Sémiologie graphique*. Paris: Mouton/Gauthier-Villars, 1967.

[2] G. Palsky, « La Sémiologie graphique de Jacques Bertin a cinquante ans », *Visions carto (en ligne)*, 2017.

[3] C. O. Wilke, *Fundamentals of data visualization: a primer on making informative and compelling figures*. O'Reilly Media, 2019.

[4] K. Dale, *Data Visualization with Python and JavaScript*. " O'Reilly Media, Inc.", 2022.